

# 遥感目标智能检测平台

基于YOLO11的遥感图像目标检测系统，支持飞机、油罐、立交桥、操场等多类目标识别

## 目录

- [项目简介](#)
- [核心亮点](#)
- [功能特性](#)
- [技术栈](#)
- [快速开始](#)
- [实训计划](#)
- [项目结构](#)
- [使用说明](#)
- [贡献指南](#)
- [许可证](#)

## 一、项目简介

遥感目标智能检测平台是一个基于深度学习的遥感图像目标检测系统，旨在实现对遥感影像中多种目标的快速、准确识别。

### 项目背景

随着遥感技术的发展，海量遥感影像数据需要高效的目标检测方法进行分析。本项目利用YOLO11深度学习模型，针对RSOD数据集进行迁移学习，实现对飞机、油罐、立交桥、操场等典型遥感目标的智能检测。

遥感图像智能解译是当前人工智能技术与地理信息科学深度融合的前沿方向。随着卫星遥感分辨率的持续提升和无人机侦察技术的普及，基于遥感图像的目标检测在多个战略性领域的价值日益凸显：

| 领域   | 典型应用              | 时效要求      |
|------|-------------------|-----------|
| 军事侦察 | 机场、港口飞机/舰船的快速定位识别 | 分钟级,时敏目标  |
| 城市规划 | 立交桥、储油罐等基础设施监测    | 周/月级      |
| 交通监测 | 高速公路、港口物流分析       | 小时级       |
| 应急救援 | 受灾区域建筑物、运动目标识别    | 实时,救援部署关键 |
| 环境监测 | 油罐泄漏、违建扩散监测       | 日级        |

### 数据集说明

- 数据集来源: RSOD（Remote Sensing Object Detection）数据集
- 图片数量: 976张遥感图像
- 标注文件: 936个XML标注文件
- 目标类别: 4类（飞机、油罐、立交桥、操场）

## 二、核心亮点

| 亮点                                                                                      | 说明                     |
|-----------------------------------------------------------------------------------------|------------------------|
|  高性能推理 | 基于YOLO11，支持半精度推理，检测速度快 |
|  多目标检测 | 支持飞机、油罐、立交桥、操场等多类目标    |
|  可视化展示 | 检测结果可视化，支持框选标注、置信度展示   |
|  批量处理  | 支持多张图片批量检测，异步任务队列      |
|  友好界面  | 现代化前端界面，操作便捷           |
|  容器化部署 | Docker一键部署，环境配置简单      |

## 三、功能特性

### 单图检测

- 支持上传单张遥感图像进行检测
- 实时返回检测结果和置信度
- 检测框可视化标注

### 批量检测

- 支持上传多张图片批量检测
- 异步任务队列处理
- 批量结果导出

### 视频检测

- 支持视频文件上传
- 逐帧目标检测
- 检测结果视频输出

### 历史记录

- 检测历史记录管理

- 支持查询和筛选
- 检测结果持久化存储



结果分析

- 检测结果结构化输出
- PDF报告生成
- 统计分析图表

四、技术栈

前端技术

| 技术           | 版本  | 说明      |
|--------------|-----|---------|
| Vue.js       | 3.x | 前端框架    |
| Element Plus | 2.x | UI组件库   |
| Axios        | 1.x | HTTP客户端 |
| ECharts      | 5.x | 数据可视化   |

后端技术

| 技术         | 版本     | 说明     |
|------------|--------|--------|
| Python     | 3.10+  | 编程语言   |
| FastAPI    | 0.100+ | Web框架  |
| PostgreSQL | 15+    | 关系型数据库 |
| Redis      | 7+     | 缓存服务   |
| MinIO      | 2023+  | 对象存储   |
| SQLAlchemy | 2.x    | ORM框架  |

AI核心技术

| 技术                 | 版本   | 说明     |
|--------------------|------|--------|
| PyTorch            | 2.0+ | 深度学习框架 |
| Ultralytics YOLO11 | 8.x  | 目标检测模型 |
| scikit-learn       | 1.3+ | 数据处理   |

# 工具链

| 工具             | 说明        |
|----------------|-----------|
| Docker         | 容器化部署     |
| Docker Compose | 多容器编排     |
| Git+GitHub     | 版本控制+代码托管 |
| Pre-commit     | 代码质量检查    |

## 五、快速开始

### 1. 环境要求

- Python 3.10+
- Docker 20.10+
- Docker Compose 2.0+
- Node.js 18+

### 2. 克隆项目

```
git clone https://github.com/your-username/rsod-web-platform.git
cd rsod-web-platform
```

### 2. 启动服务

```
# 使用Docker Compose启动所有服务
docker-compose up -d
```

### 3. 安装后端依赖

```
cd backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

### 4. 启动后端服务

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

### 5. 安装前端依赖

通过vscode 导入rsod-web-platform/frontend

```
cd frontend
# 安装基础依赖
npm install
```

6. 启动前端服务

```
npm run dev
```

7. 访问服务

- 前端地址: <http://localhost:5173>
- 后端API: <http://localhost:8000/docs>

六、实训计划

| 天数     | 阶段                | 核心内容                    | AI核心占比 | 前后端工作 |
|--------|-------------------|-------------------------|--------|-------|
| Day 1  | 环境搭建 + YOLO11基础推理 | 环境搭建、Docker部署、FastAPI集成 | 60%    | 40%   |
| Day 2  | 前端模板对接 + 单图检测     | 前后端联调、历史记录、结果展示         | 40%    | 60%   |
| Day 3  | RSOD数据集预处理 + 训练启动 | 格式转换、配置文件、启动训练          | 80%    | 20%   |
| Day 4  | 训练监控 + 批量推理       | 训练进度跟踪、批量上传API、异步队列     | 70%    | 30%   |
| Day 5  | 模型评估 + 多模型对比      | mAP计算、性能对比、报告生成         | 90%    | 10%   |
| Day 6  | 遥感场景专项优化          | 大图像切分、小目标增强、精度调优        | 80%    | 20%   |
| Day 7  | 多模式推理扩展           | 视频检测、实时流推理、前端组件         | 70%    | 30%   |
| Day 8  | 结果可视化 + PDF报告     | 检测结果结构化、PDF模板、界面美化      | 60%    | 40%   |
| Day 9  | 全流程联调 + 演示准备      | 端到端测试、性能基准、项目准备         | 50%    | 50%   |
| Day 10 | 项目答辩              | AI亮点讲解、系统演示、材料整理        | -      | -     |

七、每日任务清单

## 17 Day 1: 环境搭建 + YOLO11基础推理

- ☐ Docker Compose 部署 (PostgreSQL/Redis/MinIO)
- ☐ FastAPI 后端框架初始化
- ☐ 虚拟环境配置与依赖安装
- ☐ 前端模板引入与路由配置
- ☐ 健康检查接口测试

## 17 Day 2: 前端模板对接 + 单图检测全流程打通

- ☐ 图片上传API开发
- ☐ YOLO推理服务集成
- ☐ 检测结果返回格式定义
- ☐ 前端检测界面开发
- ☐ 历史记录列表展示

## 17 Day 3: RSOD数据集模型训练与微调

- ☐ XML标注→YOLO格式转换
- ☐ 数据集配置文件 (rsod.yaml)
- ☐ 训练脚本编写与参数配置
- ☐ 启动训练 (后台运行)
- ☐ 前端UI细节优化

## 17 Day 4: 训练监控 + 批量推理功能

- ☐ 训练进度监控
- ☐ 损失曲线分析
- ☐ 批量上传API开发
- ☐ 异步任务队列实现
- ☐ 训练日志记录

## 17 Day 5: 模型评估 + 多模型对比

- ☐ 验证集mAP计算
- ☐ 混淆矩阵生成
- ☐ 不同模型对比测试
- ☐ 训练报告生成

## 17 Day 6: 遥感场景专项优化

- ☐ 大图像切分策略
- ☐ 小目标检测增强
- ☐ 检测精度调优
- ☐ 超参数优化

## 17 Day 7: 多模式推理扩展

- ☐ 视频检测功能开发
- ☐ 实时流推理接口
- ☐ 前端视频播放组件
- ☐ 多模型切换功能

## 17 Day 8: 结果可视化 + PDF报告生成

- ☐ 检测结果结构化输出
- ☐ PDF报告模板设计
- ☐ 界面美化与交互优化
- ☐ 统计图表集成

## 17 Day 9: 全流程联调 + 演示准备

- ☐ 端到端测试
- ☐ 性能基准测试
- ☐ 演示Demo准备
- ☐ 问题排查与修复

## 17 Day 10: 项目答辩

- ☐ AI技术亮点讲解
- ☐ 系统演示
- ☐ 答辩材料整理

# 八、执行建议与风险预案

## 1. 训练并行策略

```
# 在Day3下午启动训练，可后台运行
cd backend
nohup .venv/bin/python train_model.py > train.log 2>&1 &
# 查看进度
tail -f train.log
```

## 2. 预训练模型备份

提前下载 `yolo11n.pt` 到 `backend/models/`

在 `backend/app/config.py` 中配置备用方案：

```
# 训练未完成时启用预训练模型
USE_PRETRAINED_MODEL = True
BACKUP_MODEL_PATH = "models/yolo11n_pretrained.pt"
```

### 3. 每日检查点

| 时间   | 检查内容       |
|------|------------|
| 每日上午 | 确认昨日任务完成情况 |
| 每日下午 | 进度同步与问题排查  |
| 每日结束 | 代码提交与文档更新  |

#### 4. 风险预案

| 风险   | 触发条件       | 应对措施              |
|------|------------|-------------------|
| 训练超时 | CPU训练>12小时 | 使用预训练模型，后续继续训练    |
| 环境问题 | 依赖安装失败     | 使用Docker镜像，或预配置环境 |
| 功能延期 | 某功能开发超时    | 优先保证答辩核心功能        |

## 九、项目结构

```
rsod-web-platform/                                     # 项目根目录
├── backend/                                           # 后端代码目录
│   ├── app/                                           # 应用核心模块
│   │   ├── api/                                       # API路由层
│   │   │   ├── __init__.py                          # 路由包初始化
│   │   │   └── detection.py                         # 目标检测API接口
│   │   ├── models/                                   # 数据模型层
│   │   │   ├── __init__.py
│   │   │   ├── database.py                         # 数据库连接配置
│   │   │   └── schemas.py                          # Pydantic数据模型
│   │   ├── services/                                # 业务逻辑层
│   │   │   ├── __init__.py
│   │   │   ├── detection_service.py                # YOLO检测服务
│   │   │   ├── minio_service.py                    # MinIO对象存储服务
│   │   │   └── redis_service.py                    # Redis缓存服务
│   │   ├── utils/                                   # 工具函数层
│   │   │   ├── __init__.py
│   │   │   └── file_utils.py                       # 文件操作工具
│   │   ├── __init__.py
│   │   └── config.py                                # 配置管理
│   └── models/                                       # 模型文件目录
```



```

├── yolol1n.pt                # YOLO11n预训练模型
├── rsod_yolol1n/            # 自定义训练模型
│   ├── weights/
│   │   ├── best.pt          # 最佳权重
│   │   └── last.pt          # 最后权重
├── data/                    # 数据存储目录
│   ├── rsod/                # RSOD数据集
│   │   ├── images/          # 原始遥感图像
│   │   │   ├── aircraft_*.jpg # 飞机类图像(446张)
│   │   │   ├── oiltank_*.jpg  # 油罐类图像(165张)
│   │   │   ├── overpass_*.jpg # 立交桥类图像(176张)
│   │   │   └── playground_*.jpg # 操场类图像(149张)
│   │   ├── annotations/      # XML标注文件(PASCAL VOC格式)
│   │   │   ├── aircraft_*.xml
│   │   │   ├── oiltank_*.xml
│   │   │   ├── overpass_*.xml
│   │   │   └── playground_*.xml
│   │   └── yolo_dataset/      # YOLO格式数据集
│   │       ├── images/        # 图像文件
│   │       │   ├── train/      # 训练集(~749张)
│   │       │   │   └── val/      # 验证集(~187张)
│   │       │   ├── labels/      # 标注文件(YOLO txt格式)
│   │       │   │   ├── train/    # 训练集标签
│   │       │   │   └── val/      # 验证集标签
│   │       └── rsod.yaml        # 数据集配置文件
├── uploads/                  # 上传文件临时目录
│   └── [临时上传的图片文件]
├── results/                  # 检测结果存储目录
│   └── [检测结果图像和JSON]
├── train_model.py            # 模型训练脚本
├── convert_rsod.py           # 数据集格式转换脚本
├── requirements.txt           # Python依赖列表
├── main.py                   # FastAPI应用入口
├── .venv/                    # Python虚拟环境
├── .env                       # 环境变量配置
├── .env.example              # 环境变量配置示例
├── frontend/                 # 前端代码目录
│   ├── public/               # 静态公共资源
│   │   ├── favicon.ico       # 网站图标
│   │   └── index.html         # HTML模板
│   ├── src/                  # 前端源代码
│   │   ├── api/              # API接口层
│   │   │   ├── detection.ts   # 检测相关API
│   │   │   ├── history.ts     # 历史记录API
│   │   │   └── upload.ts      # 文件上传API
│   │   ├── assets/           # 前端资源
│   │   │   ├── images/        # 图片资源
│   │   │   └── styles/         # 样式文件
│   │   │       └── main.css    # 全局样式
│   │   ├── components/        # Vue组件
│   │   └── common/            # 通用组件

```



| 目录                    | 说明               |
|-----------------------|------------------|
| backend/app/api/      | API路由定义，处理HTTP请求 |
| backend/app/models/   | 数据模型和数据库结构       |
| backend/app/services/ | 核心业务逻辑服务         |
| backend/app/utils/    | 通用工具函数           |
| backend/models/       | YOLO模型权重文件       |
| backend/data/rsod/    | RSOD遥感数据集        |
| backend/uploads/      | 临时上传文件           |
| backend/results/      | 检测结果输出           |

前端目录结构

| 目录                       | 说明             |
|--------------------------|----------------|
| frontend/src/api/        | 封装API调用接口      |
| frontend/src/components/ | Vue组件（可复用UI组件） |
| frontend/src/views/      | 页面级组件（路由页面）    |
| frontend/src/router/     | Vue Router路由配置 |
| frontend/src/store/      | Pinia状态管理      |

数据存储目录结构

| 目录                | 说明    | 存储内容             |
|-------------------|-------|------------------|
| storage/minio/    | 对象存储  | 用户上传图片、检测结果      |
| storage/postgres/ | 关系数据库 | 结构化数据（用户信息、检测记录） |
| storage/redis/    | 缓存数据库 | 会话缓存、热点数据        |

MinIO存储桶结构

```
rsod-bucket/                                # 主存储桶
├── images/                                  # 原始上传图片
│   ├── {user_id}/
│   │   ├── {timestamp}_{filename}
│   │   └── ...
├── results/                                # 检测结果图片
│   ├── {user_id}/
│   │   ├── {timestamp}_result.jpg
│   │   └── ...
└── models/                                 # 模型文件备份
    └── yolol1n.pt
```

## PostgreSQL数据库表结构

```
-- 检测记录表
CREATE TABLE detection_records (
    id SERIAL PRIMARY KEY,
    user_id VARCHAR(50),
    image_path TEXT,
    result_json JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 用户表
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(100) UNIQUE,
    password_hash VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

## 十、使用说明

### 模型训练

```
cd backend
source .venv/bin/activate
python train_model.py
```

### 数据转换

```
cd backend
source .venv/bin/activate
python convert_rsod.py
```

### API接口

## 单图检测接口

```
POST /api/detection/single
Content-Type: multipart/form-data
```

参数:

- file: 图片文件
- model\_name: 模型名称(可选)

返回:

```
{
  "success": true,
  "message": "检测成功",
  "result": {
    "boxes": [...],
    "confidences": [...],
    "class_names": [...]
  }
}
```